MATH550 | HOMEWORK #2

Jack Hickisch

November 2, 2026

Abstract

This report assumes familiarity with the assignment as provided by Dr. Sprinkle. Most problems only include a plot and brief description as requested.

1 Problem 1: Poisson Equation

This problem was completed and as it provides no interesting insights beyond what was already discussed in HW 0, I won't present any findings.

2 Problem 2: Poisson Equation with an Immersed Boundary

When experimenting with different spacing for the imersed boundary points I came to the following conclusions:

1. When the IB points are very far apart I got a better solution at coarser discretizations and faster convergence with accuracy being scarified at finer discretizations.
2. When the IB points are closer together the code runs much slower and does not produce a better result. In fact, this causes scaling differences in the matrices such that the system of linear equations we are solving is near-singular.
3. Thus, it is best to use grid spacing on the immersed boundary that is similar to the background grid.

When using GMRES, my linear system system had no trouble converging with the default tolerance of $tol = 1 * 10^{-6}$. I imagine this is not an issue for me because of how I setup up boundary conditions in the matrix, although I'm not sure. I'd have to look into this more since everything else in the system should be the same as any other code for the same problem. If I didn't need this tolerance I could reduce it for improved efficiency.

Note on AI usage: I got through most of this problem without the use of AI but at the end I used it to find a few bugs. My code ran on it's own without the use of AI and converged to a sensible but incorrect solution. They are listed below:

1. I had a mix of solutions for u in my code that was my primary error. Between HW #0 and HW #2, the sine and cosine functions "flipped" and I didn't change one of them. AI actually thought I was solving for a different solution and thought all my correct parts were wrong and incorrect parts were right since I was inconsistent with the PDE I was solving. In HW #0, $u(x,y) = \sin(f(x))\cos(f(y))$ where here we had $u(x,y) = \sin(f(y))\cos(f(x))$
2. I pre-allocated the size of one of my matrices incorrectly but that didn't matter since MATLAB doesn't care, it just gets slower.
3. My Σ in my 2D gaussian function was wrong even though I asked AI what the function for a 2D Gaussian was so either it was wrong at first or I copied it down wrong originally.
4. I'm not sure if there is an error in the assignment but I guess the $\mathscr{S}$ matrix must also be scaled by Δx^2 which I didn't see in the given equation $\nabla^2 u = f(x,y) + \sum_{k=1}^{N_i b} q_k \delta_a\left(x - x_k^{ib}, y - y_k^{ib}\right) = f(x,y) + \mathscr{S}^{ib}(\boldsymbol{q})$ but that made my solution work so I implemented it.

The plot and convergence for Problem 2 is shown below. The solution is converging at the rate expected for an immersed boundary as discussed in the problem description.

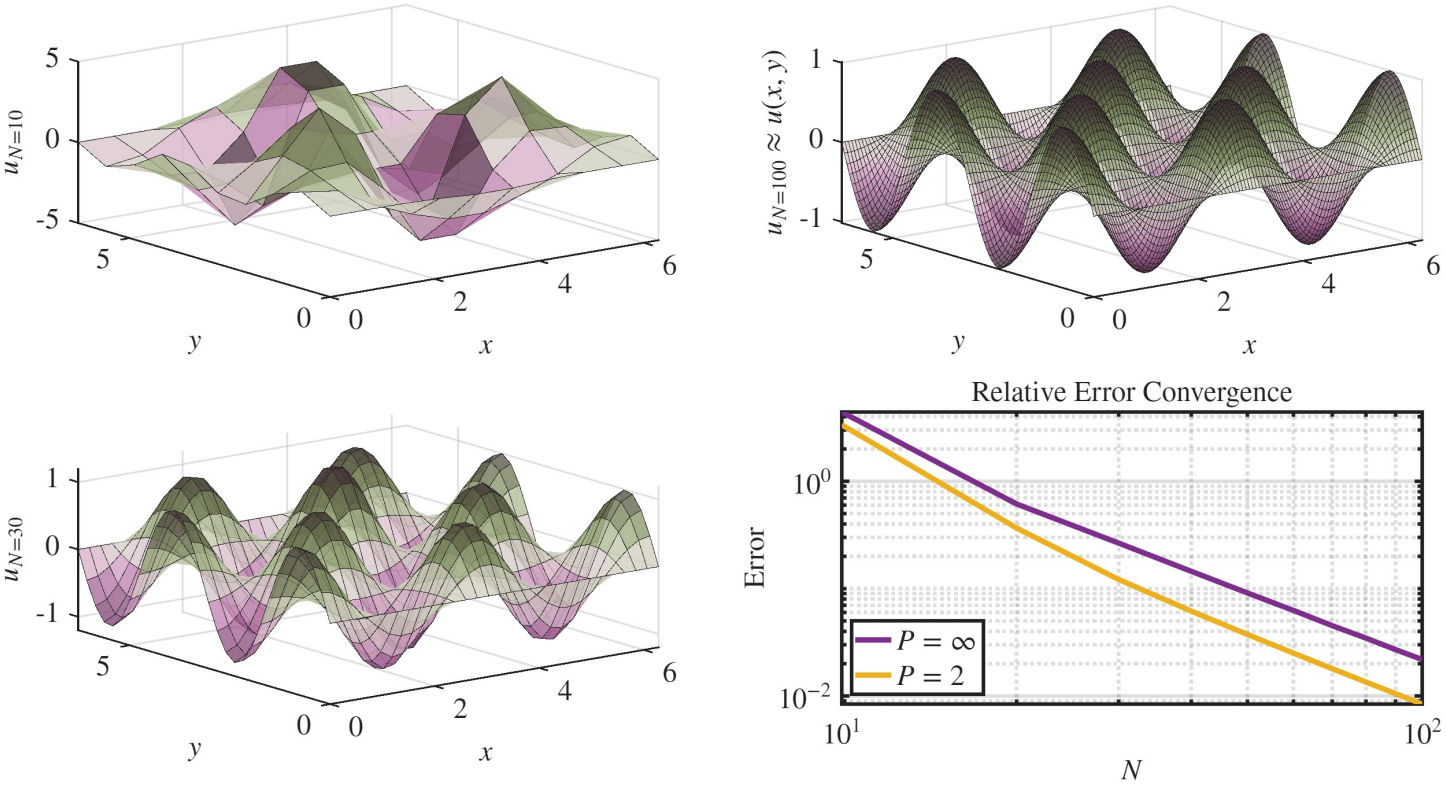


Figure 1: Problem 2 Plot

3 Problem 3: An Almost Minimal Surface

Only a few lines needed to be changed to solve the problem described. The plot for Problem 3 is shown below:

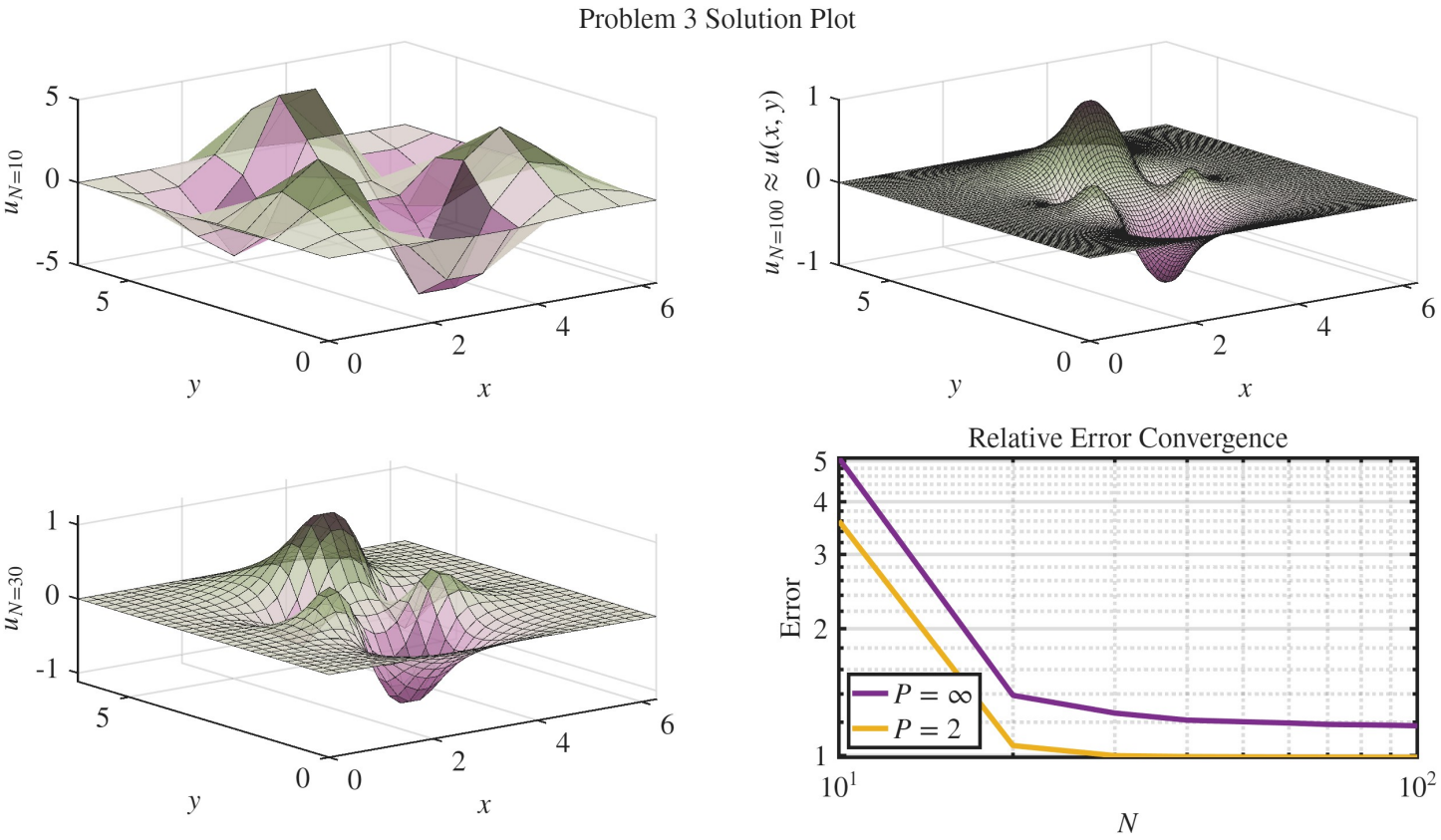


Figure 2: Problem 3 Plot

4 Problem 4: Charlie's Rule

I had a kinda crazy idea for this that I implemented with the help of Google Gemini Pro 3.1. Gemini wrote nearly all this code starting from where I was at and modifying things as directed by me to make this work. I'm not sure everything is totally robust but it seemed to work. The idea of a minimal surface (which this isn't quite but close enough) inspired me to try and make immersed boundaries for a PDE that are elevation contours. I chose to try and recreate the rough shape of Longs Peak by first importing the USGS elevation data and selecting some elevation contours within a boundary to use. Then to smooth these out (just because I can) I used Fourier Epicycles to generate smooth curves for the elevation contours. Then I set 4 immersed boundaries as elevation contours to try and hit and AI had the good idea to add Tikhonov regularization to reduce the spikiness of the solution that is heavily constrained by penalizing large values to the solution. Gemini also implemented some of the suggestions in the assignment for me to speed up the code for this. The result is kinda fun, see what you think!

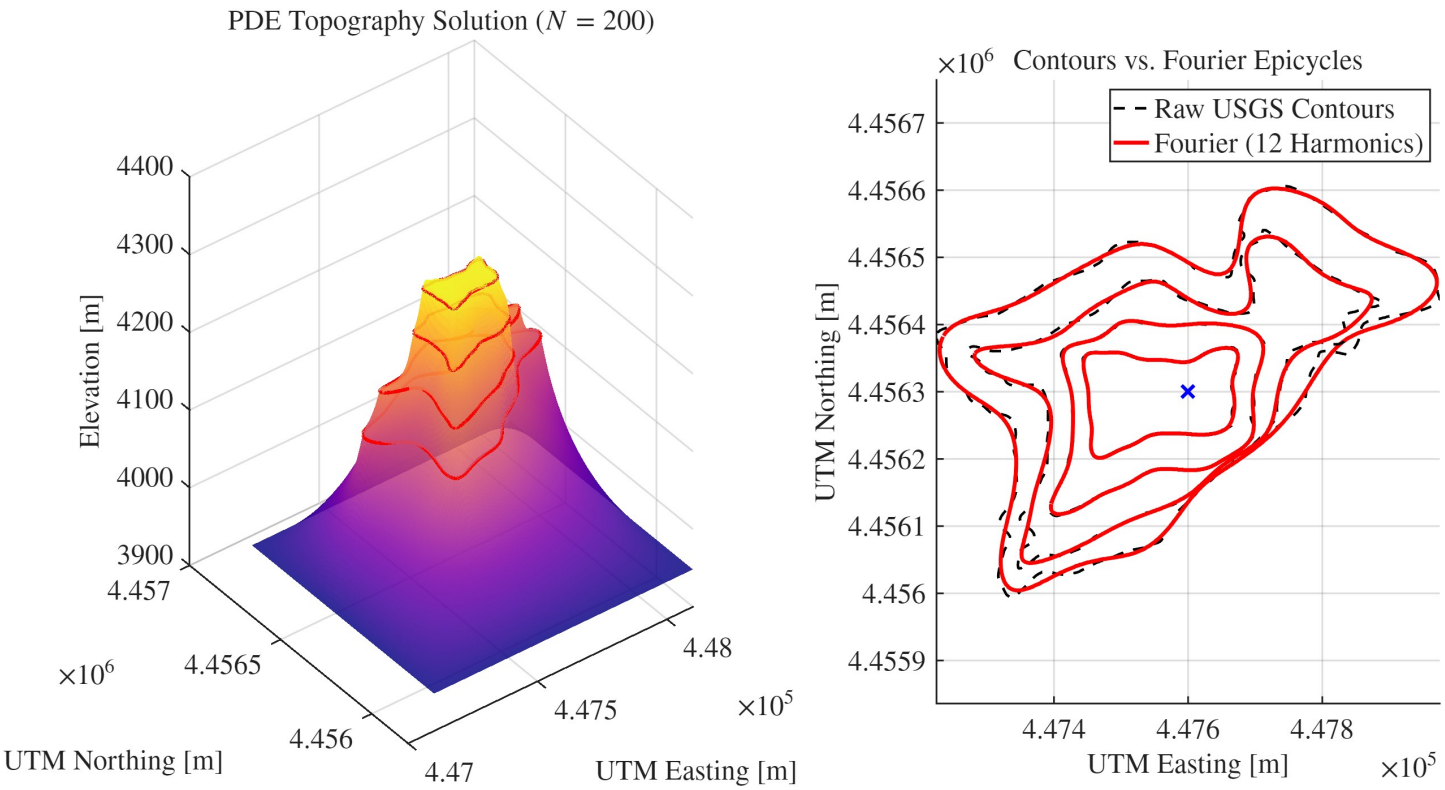


Figure 3: Longs Peak PDE!



Figure 4: Longs Peak viewed in Gaia GPS 3D (gpx track in pink)

5 Commented Code

Commented MATLAB Code is provided below.

5.1 Problems 1 & 2

```
clc, clear, close all
```

```
font = 24; %fontsize
lw = 4.5; %linewidth
lwlrg = 6; %linewidth large
lwsml = 3; %linewidth small
window_size_for_export = [0, 50, 1920, 975];

a_s = 1.2; % "a scalar" which is variance scalar for 2D Gaussian
[xdomain,ydomain] = deal([0,2*pi]);
N_range = [10, 100];
N_plot = [10, 30, 100];
N = linspace(N_range(1),N_range(2),(N_range(2)-N_range(1))/10+1); % grid sizes
h = (xdomain(2)-xdomain(1))./(N-1);                               % step sizes
[x,y,X,Y,theta,xib,yib,Sigma] = deal(cell(length(N),1));          % setup x
   scale for each N

[Lap, Sib, Jib, LHSMat, RHSVec, ResVec, q, f, f2, u, u_actual] = deal(cell(
   length(N),1)); % discrete derivative matrices and f for each N and solution
   matrix

for n = 1:length(N)
   Lap{n} = sparse(N(n)^2);
   f{n} = zeros(N(n)^2,1);
   x{n} = linspace(xdomain(1),xdomain(2),N(n));
   y{n} = linspace(ydomain(1),ydomain(2),N(n));
   [X{n},Y{n}] = meshgrid(x{n},y{n});
   u_actual{n} = zeros(length(x{n}));
   for j = 1:length(y{n})
       u_actual{n}(j,:) = cos(2*x{n})*sin(2*y{n}(j));
   end
   theta{n} = linspace(0,2*pi,floor(2*pi/h(n)));
   xib{n} = pi + cos(theta{n});
   yib{n} = pi + sin(theta{n});
   xib{n} = xib{n}(1:end-1); % 0 = 2*pi and only need one coordinate
   yib{n} = yib{n}(1:end-1); % 0 = 2*pi and only need one coordinate
   Sigma{n} = [(a_s*h(n))^2,0;0,(a_s*h(n))^2];
   f2{n} = zeros(length(xib{n}),1);
end

for n = 1:length(N)
```

```
    Lap_row = 1;
    for j = 1:length(y{n})
        for i = 1:length(x{n})
            % Laplacian Computation
            if i == 1 || j == 1 || i == length(x{n}) || j == length(y{n}) % on
               boundary
                Lap{n}(Lap_row, global_index(j,i,N(n))) = -8*h(n)^2;
            else
                Lap{n}(Lap_row, global_index(j  ,i  ,N(n))) = -4;
                Lap{n}(Lap_row, global_index(j  ,i+1,N(n))) =  1;
                Lap{n}(Lap_row, global_index(j+1,i  ,N(n))) =  1;
                Lap{n}(Lap_row, global_index(j  ,i-1,N(n))) =  1;
                Lap{n}(Lap_row, global_index(j-1,i  ,N(n))) =  1;
            end
            f{n}(Lap_row) = -8*h(n)^2*cos(2*x{n}(i))*sin(2*y{n}(j));
            %f{n}(Lap_row) = 0;

            % Sib Computataion
            Sib{n}(Lap_row,:) = s_ib(x{n}(i),y{n}(j),xib{n},yib{n},Sigma{n},h(n
               ));

            Lap_row = Lap_row + 1;
        end
    end

    for i = 1:length(xib{n})
        Jib{n}(i,:) = j_ib(x{n},y{n},xib{n}(i),yib{n}(i),Sigma{n},h(n));
        f2{n}(i) = cos(2*xib{n}(i))*sin(2*yib{n}(i));
    end

    LHSMat{n} = [Lap{n},-Sib{n};Jib{n},zeros(length(xib{n}))];
    RHSVec{n} = [f{n};f2{n}];

    % ResVec{n} = LHSMat{n}\RHSVec{n};
    ResVec{n} = gmres(LHSMat{n},RHSVec{n},1000,1e-6,1);
    u{n} = reshape(ResVec{n}(1:N(n)^2),N(n),N(n))';
    q{n} = ResVec{n}(N(n)^2+1:end);
end

function delta_a = s_ib(x,y,xib,yib,Sigma,h)
```

```
    delta_a = zeros(1,length(xib));
    for i = 1:length(delta_a)
        delta_a(i) = (2*pi*sqrt(det(Sigma)))^-1*exp(-0.5*([xib(i);yib(i)]-[x;y
           ])'*Sigma^-1*([xib(i);yib(i)]-[x;y]))*h^2; % used AI to find 2D
           Gaussian equation
    end
end


function j_res = j_ib(x,y,xib,yib,Sigma,h)
    j_res = zeros(1,length(x)*length(y));

    u_row = 1;
    for j = 1:length(y)
        for i = 1:length(x)
            j_res(u_row) = (2*pi*sqrt(det(Sigma)))^-1*exp(-0.5*([xib;yib]-[x(i)
               ;y(j)])'*Sigma^-1*([xib;yib]-[x(i);y(j)]))*h^2;
            u_row = u_row + 1;
        end
    end
end

function g = global_index(j, i, N)
    g = N*(j-1) + i;
end

figure(1)
i=1;
plots_out_of_order = [1,3,2];
z_label = [strcat("$u_{N=", num2str(N_plot(1)), "}$"), strcat("$u_{N=", num2str
   (N_plot(2)), "}$"), strcat("$u_{N=", num2str(N_plot(3)), "} \approx u(x,y)
   $")];
for n = N_plot
    subplot(2,2,plots_out_of_order(i))
    plot_i = find(N == n);
    surf(X{plot_i},Y{plot_i},u{plot_i},FaceAlpha=0.75)
    colormap(slanCM('bamo'));
    xlabel('$x$', 'Interpreter', 'latex');
    ylabel('$y$', 'Interpreter', 'latex');
    zlabel(z_label(i), 'Interpreter', 'latex');
```

```
    % xticks(0:1:5)
    % yticks(0:1:5)
    % zticks(-1:0.5:1)
    fontsize(font,"points")
    xlim([0,2*pi])
    ylim([0,2*pi])
    % zlim([-1,1])
    set(gca, 'TickLabelInterpreter', 'latex')
    ax1 = gca;
    ax1.LineWidth = lwsml*.5;
    i = i + 1;
end



%% Error calcs

[Err_p_2,Err_p_inf] = deal(zeros(length(N),1)); % setup arrays to hold errors

for j = 1:length(N)
    top_diff = u_actual{j} - u{j}; % top of error equation
    top_diff = top_diff(:); %flattening
    Err_p_2(j) = norm(top_diff)/norm(u_actual{j}(:)); %P-2 Norm Error
    Err_p_inf(j) = max(abs(top_diff))/max(abs(u_actual{j}(:))); %P-inf Norm
       Error
end

sgtitle({'Analytic and Finite Difference Solutions to Poisson Equation $\nabla
   ^2 u = f(x,y)$ with the Test Solution $u(x,y)=\cos(2x)\sin(2y)$', 'and a
   Circular Dirichelet Imersed Boundary set at Nominal Solution'}, 'Interpreter
   ', 'latex');

subplot(2,2,4)
loglog(N,Err_p_inf,LineWidth=lw,Color='#7E2F8E')
hold on
loglog(N,Err_p_2,LineWidth=lw,Color='#EDB120')
% axis square
legend('$P=\infty$','$P=2$','Interpreter', 'latex',Location='southwest')
xlabel('$N$', 'Interpreter', 'latex');
ylabel('Error', 'Interpreter', 'latex');
```

```
title('Relative Error Convergence', 'Interpreter', 'latex');
fontsize(font,"points")
grid on
box on
set(gca, 'TickLabelInterpreter', 'latex')
ax2 = gca;
ax2.LineWidth = lwsml;

set(gcf, 'Position', window_size_for_export);
exportgraphics(gcf,"HW2_1and2.jpg")
```

5.2 Problem 3

```
clc, clear, close all

font = 24; %fontsize
lw = 4.5; %linewidth
lwlrg = 6; %linewidth large
lwsml = 3; %linewidth small
window_size_for_export = [0, 50, 1920, 975];

a_s = 1.2; % "a scalar" which is variance scalar for 2D Gaussian
[xdomain,ydomain] = deal([0,2*pi]);
N_range = [10, 100];
N_plot = [10, 30, 100];
N = linspace(N_range(1),N_range(2),(N_range(2)-N_range(1))/10+1); % grid sizes
h = (xdomain(2)-xdomain(1))./(N-1);                               % step sizes
[x,y,X,Y,theta,xib,yib,Sigma] = deal(cell(length(N),1));          % setup x
   scale for each N

[Lap, Sib, Jib, LHSMat, RHSVec, ResVec, q, f, f2, u, u_actual] = deal(cell(
   length(N),1)); % discrete derivative matrices and f for each N and solution
   matrix

for n = 1:length(N)
   Lap{n} = sparse(N(n)^2);
   f{n} = zeros(N(n)^2,1);
   x{n} = linspace(xdomain(1),xdomain(2),N(n));
   y{n} = linspace(ydomain(1),ydomain(2),N(n));
   [X{n},Y{n}] = meshgrid(x{n},y{n});
   u_actual{n} = zeros(length(x{n}));
```

```
    for j = 1:length(y{n})
        u_actual{n}(j,:) = cos(2*x{n})*sin(2*y{n}(j));
    end
    theta{n} = linspace(0,2*pi,floor(2*pi/h(n)));
    xib{n} = pi + cos(theta{n});
    yib{n} = pi + sin(theta{n});
    xib{n} = xib{n}(1:end-1); % 0 = 2*pi and only need one coordinate
    yib{n} = yib{n}(1:end-1); % 0 = 2*pi and only need one coordinate
    Sigma{n} = [(a_s*h(n))^2,0;0,(a_s*h(n))^2];
    f2{n} = zeros(length(xib{n}),1);
end

for n = 1:length(N)
    Lap_row = 1;
    for j = 1:length(y{n})
        for i = 1:length(x{n})
            % Laplacian Computation
            if i == 1 || j == 1 || i == length(x{n}) || j == length(y{n}) % on
               boundary
                Lap{n}(Lap_row, global_index(j,i,N(n))) = -8*h(n)^2;
            else
                Lap{n}(Lap_row, global_index(j  ,i  ,N(n))) = -4;
                Lap{n}(Lap_row, global_index(j  ,i+1,N(n))) =  1;
                Lap{n}(Lap_row, global_index(j+1,i  ,N(n))) =  1;
                Lap{n}(Lap_row, global_index(j  ,i-1,N(n))) =  1;
                Lap{n}(Lap_row, global_index(j-1,i  ,N(n))) =  1;
            end
            % f{n}(Lap_row) = -8*h(n)^2*cos(2*x{n}(i))*sin(2*y{n}(j));
            f{n}(Lap_row) = 0;

            % Sib Computataion
            Sib{n}(Lap_row,:) = s_ib(x{n}(i),y{n}(j),xib{n},yib{n},Sigma{n},h(n
               ));

            Lap_row = Lap_row + 1;
        end
    end

    for i = 1:length(xib{n})
        Jib{n}(i,:) = j_ib(x{n},y{n},xib{n}(i),yib{n}(i),Sigma{n},h(n));
```

```
        f2{n}(i) = cos(2*xib{n}(i))*sin(2*yib{n}(i));
    end

    LHSMat{n} = [Lap{n},-Sib{n};Jib{n},zeros(length(xib{n}))];
    RHSVec{n} = [f{n};f2{n}];

    % ResVec{n} = LHSMat{n}\RHSVec{n};
    ResVec{n} = gmres(LHSMat{n},RHSVec{n},1000,1e-6,1);
    u{n} = reshape(ResVec{n}(1:N(n)^2),N(n),N(n))';
    q{n} = ResVec{n}(N(n)^2+1:end);
end

function delta_a = s_ib(x,y,xib,yib,Sigma,h)
    delta_a = zeros(1,length(xib));
    for i = 1:length(delta_a)
        delta_a(i) = (2*pi*sqrt(det(Sigma)))^-1*exp(-0.5*([xib(i);yib(i)]-[x;y
           ])'*Sigma^-1*([xib(i);yib(i)]-[x;y]))*h^2; % used AI to find 2D
           Gaussian equation
    end
end


function j_res = j_ib(x,y,xib,yib,Sigma,h)
    j_res = zeros(1,length(x)*length(y));

    u_row = 1;
    for j = 1:length(y)
        for i = 1:length(x)
            j_res(u_row) = (2*pi*sqrt(det(Sigma)))^-1*exp(-0.5*([xib;yib]-[x(i)
               ;y(j)])'*Sigma^-1*([xib;yib]-[x(i);y(j)]))*h^2;
            u_row = u_row + 1;
        end
    end
end

function g = global_index(j, i, N)
    g = N*(j-1) + i;
end

figure(1)
```

```
i=1;
plots_out_of_order = [1,3,2];
z_label = [strcat("$u_{N=", num2str(N_plot(1)), "}$"), strcat("$u_{N=", num2str
   (N_plot(2)), "}$"), strcat("$u_{N=", num2str(N_plot(3)), "} \approx u(x,y)
   $")];
for n = N_plot
    subplot(2,2,plots_out_of_order(i))
    plot_i = find(N == n);
    surf(X{plot_i},Y{plot_i},u{plot_i},FaceAlpha=0.75)
    colormap(slanCM('bamo'));
    xlabel('$x$', 'Interpreter', 'latex');
    ylabel('$y$', 'Interpreter', 'latex');
    zlabel(z_label(i), 'Interpreter', 'latex');
    % xticks(0:1:5)
    % yticks(0:1:5)
    % zticks(-1:0.5:1)
    fontsize(font,"points")
    xlim([0,2*pi])
    ylim([0,2*pi])
    % zlim([-1,1])
    set(gca, 'TickLabelInterpreter', 'latex')
    ax1 = gca;
    ax1.LineWidth = lwsml*.5;
    i = i + 1;
end



%% Error calcs

[Err_p_2,Err_p_inf] = deal(zeros(length(N),1)); % setup arrays to hold errors

for j = 1:length(N)
    top_diff = u_actual{j} - u{j}; % top of error equation
    top_diff = top_diff(:); %flattening
    Err_p_2(j) = norm(top_diff)/norm(u_actual{j}(:)); %P-2 Norm Error
    Err_p_inf(j) = max(abs(top_diff))/max(abs(u_actual{j}(:))); %P-inf Norm
       Error
end
```

```
sgtitle('Problem 3 Solution Plot', 'Interpreter', 'latex');

subplot(2,2,4)
loglog(N,Err_p_inf,LineWidth=lw,Color='#7E2F8E')
hold on
loglog(N,Err_p_2,LineWidth=lw,Color='#EDB120')
% axis square
legend('$P=\infty$','$P=2$','Interpreter', 'latex',Location='southwest')
xlabel('$N$', 'Interpreter', 'latex');
ylabel('Error', 'Interpreter', 'latex');
title('Relative Error Convergence', 'Interpreter', 'latex');
fontsize(font,"points")
grid on
box on
set(gca, 'TickLabelInterpreter', 'latex')
ax2 = gca;
ax2.LineWidth = lwsml;

set(gcf, 'Position', window_size_for_export);
exportgraphics(gcf,"HW2_3.jpg")
```

5.3 Problem 4 (mostly AI Generated)

```
clc; clear; close all;

%% 1. DOWNLOAD LONGS PEAK TOPOGRAPHY DATA
fprintf('Downloading Longs Peak elevation data from USGS...\n');

% Longs Peak Center in UTM Zone 13N (Meters)
utm_easting_center  = 447600; % Easting (m)
utm_northing_center = 4456300; % Northing (m)

lat_min = 40.2505; lat_max = 40.2595;
lon_min = -105.6209; lon_max = -105.6093;

temp_tif = [tempname, '.tif'];
try
    websave(temp_tif, sprintf(['https://elevation.nationalmap.gov/arcgis/rest/
       services/' ...
        '3DEPElevation/ImageServer/exportImage?bbox=%.4f,%.4f,%.4f,%.4f' ...
        '&bboxSR=4326&size=200,200&format=tiff&f=image'], lon_min, lat_min,
```

```
            lon_max, lat_max));
    % Read raster directly. Imread returns Z(row, col) = Z(y, x).
    Z_elevation = double(imread(temp_tif));
    delete(temp_tif);
catch
    fprintf('USGS API unreachable. Generating synthetic Longs Peak profile...\n
       ');
    [Xgrid, Ygrid] = meshgrid(linspace(-1,1,200));
    Z_elevation = 4346 * exp(-2*(Xgrid.^2 + Ygrid.^2)) + 50*cos(8*Xgrid).*sin
       (8*Ygrid);
end

%% 2. EXTRACT STRICTLY CLOSED CONTOURS
Z_smooth = imgaussfilt(Z_elevation, 1.5);
z_min = min(Z_smooth(:));
z_max = max(Z_smooth(:));

num_contours = 4; % Changed to 3 contours
[x_raw, y_raw, x_scaled, y_scaled] = deal(cell(num_contours, 1));
margin = 3.0;

% STEP A: Find lowest closed contour
valid_base_found = false;
test_frac = 0.20;
while test_frac < 0.95 && ~valid_base_found
    z_test = z_min + test_frac*(z_max - z_min);
    C = contourc(1:size(Z_smooth,2), 1:size(Z_smooth,1), Z_smooth, [z_test
       z_test]);

    idx = 1; longest_len = 0; best_contour = [];
    while idx < size(C,2)
        num_pts = C(2, idx);
        segment = C(:, idx+1 : idx+num_pts);

        if min(segment(1,:)) > margin && max(segment(1,:)) < size(Z_smooth,2) -
            margin && ...
           min(segment(2,:)) > margin && max(segment(2,:)) < size(Z_smooth,1) -
               margin
            if num_pts > longest_len
                longest_len = num_pts;
```

```
                best_contour = segment;
            end
        end
        idx = idx + num_pts + 1;
    end

    if ~isempty(best_contour)
        valid_base_found = true;
        base_frac = test_frac;
    else
        test_frac = test_frac + 0.02;
    end
end

% STEP B: Distribute elevations evenly
top_frac = min(0.95, base_frac + 0.4);
if (top_frac - base_frac) < 0.15, top_frac = base_frac + 0.15; end
fractions = linspace(base_frac, top_frac, num_contours);
target_elevations_m = zeros(num_contours, 1);

for c = 1:num_contours
    z_test = z_min + fractions(c)*(z_max - z_min);
    target_elevations_m(c) = z_test; % True physical elevation in meters

    C = contourc(1:size(Z_smooth,2), 1:size(Z_smooth,1), Z_smooth, [z_test
       z_test]);
    idx = 1; longest_len = 0; best_contour = [];
    while idx < size(C,2)
        num_pts = C(2, idx);
        segment = C(:, idx+1 : idx+num_pts);

        if min(segment(1,:)) > margin && max(segment(1,:)) < size(Z_smooth,2) -
            margin && ...
           min(segment(2,:)) > margin && max(segment(2,:)) < size(Z_smooth,1) -
               margin
            if num_pts > longest_len
                longest_len = num_pts;
                best_contour = segment;
            end
        end
```

```
        idx = idx + num_pts + 1;
    end

    x_c = best_contour(1,:)'; y_c = best_contour(2,:)';
    if norm([x_c(1)-x_c(end), y_c(1)-y_c(end)]) > 1e-3
        x_c(end+1) = x_c(1); y_c(end+1) = y_c(1);
    end
    x_raw{c} = x_c; y_raw{c} = y_c;
end

scaled_elevs = target_elevations_m;

% STEP C: Scale domain globally across ALL contours
all_x = cat(1, x_raw{:}); all_y = cat(1, y_raw{:});
center_x = (min(all_x) + max(all_x)) / 2;
center_y = (min(all_y) + max(all_y)) / 2;

max_radius = max(max(abs(all_x - center_x)), max(abs(all_y - center_y)));
scale_factor = (pi * 0.75) / max_radius;

for c = 1:num_contours
    x_scaled{c} = pi + (x_raw{c} - center_x) * scale_factor;
    y_scaled{c} = pi + (y_raw{c} - center_y) * scale_factor;
end

%% 3 & 4. FOURIER CONVERSION & IB PDE SETUP
font = 24; lw = 4.5; lwlrg = 6; lwsml = 3;
window_size_for_export = [0, 50, 1920, 975];
a_s = 1.2;
[xdomain, ydomain] = deal([0, 2*pi]);

N = 200;
h = (xdomain(2) - xdomain(1)) / (N - 1);
x = linspace(xdomain(1), xdomain(2), N);
y = linspace(ydomain(1), ydomain(2), N);
[X, Y] = meshgrid(x, y);

% Map PDE grid [0, 2*pi] to 1km x 1km UTM coordinates
x_utm = utm_easting_center  + (x - pi) * (1000 / (2*pi));
y_utm = utm_northing_center + (y - pi) * (1000 / (2*pi));
```

```
[X_utm, Y_utm] = meshgrid(x_utm, y_utm);

theta = linspace(0, 2*pi, max(400, floor(10*pi/h)));
num_harmonics = 12;

xib = []; yib = []; f2 = [];
xib_plot = cell(num_contours, 1); yib_plot = cell(num_contours, 1);
xib_utm  = cell(num_contours, 1); yib_utm  = cell(num_contours, 1);

for c = 1:num_contours
    [xc, yc, a_cf, b_cf, c_cf, d_cf] = contour_to_fourier(x_scaled{c}, y_scaled
       {c}, num_harmonics);
    [x_full, y_full] = generate_fourier_epicycle(theta, xc, yc, a_cf, b_cf,
       c_cf, d_cf);

    xib_plot{c} = x_full(1:end-1)'; yib_plot{c} = y_full(1:end-1)';
    xib = [xib; xib_plot{c}]; yib = [yib; yib_plot{c}];

    xib_utm{c} = utm_easting_center  + (xib_plot{c} - pi) * (1000 / (2*pi));
    yib_utm{c} = utm_northing_center + (yib_plot{c} - pi) * (1000 / (2*pi));

    f2 = [f2; ones(length(xib_plot{c}), 1) * scaled_elevs(c)];
end

%% 5. FAST VECTORIZED MATRIX ASSEMBLY
fprintf('Assembling Matrices...\n');
tic;

row_L = zeros(N^2 * 5, 1); col_L = zeros(N^2 * 5, 1); val_L = zeros(N^2 * 5, 1)
   ;
idx_L = 1;
for j = 1:N
    for i = 1:N
        g = global_index(j, i, N);
        if i == 1 || j == 1 || i == N || j == N
            row_L(idx_L) = g; col_L(idx_L) = g;
            val_L(idx_L) = 1 / h^2;
            idx_L = idx_L + 1;
        else
            row_L(idx_L:idx_L+4) = g;
```

```
            col_L(idx_L:idx_L+4) = [g, global_index(j, i+1, N), global_index(j,
                i-1, N), ...
                                    global_index(j+1, i, N), global_index(j
                                        -1, i, N)];
            val_L(idx_L:idx_L+4) = [-4, 1, 1, 1, 1] / h^2;
            idx_L = idx_L + 5;
        end
    end
end
Lap = sparse(row_L(1:idx_L-1), col_L(1:idx_L-1), val_L(1:idx_L-1), N^2, N^2);

row_S = []; col_S = []; val_S = [];
r_cut = 4 * a_s * h;
const_factor = 1 / (2 * pi * (a_s*h)^2) * h^2;
X_vec = X(:); Y_vec = Y(:);

for k = 1:length(xib)
    idx = find(abs(X_vec - xib(k)) <= r_cut & abs(Y_vec - yib(k)) <= r_cut);
    if ~isempty(idx)
        dx = xib(k) - X_vec(idx); dy = yib(k) - Y_vec(idx);
        dist_sq = (dx.^2 + dy.^2) / (a_s*h)^2;
        vals = const_factor * exp(-0.5 * dist_sq);

        row_S = [row_S; idx];
        col_S = [col_S; repmat(k, length(idx), 1)];
        val_S = [val_S; vals];
    end
end
Sib = sparse(row_S, col_S, val_S, N^2, length(xib));
Jib = sparse(col_S, row_S, val_S, length(xib), N^2);

% Set base boundary condition on domain exterior
RHS_Lap = zeros(N^2, 1);
base_elevation_m = min(target_elevations_m) - 200;
for j = 1:N
    for i = 1:N
        if i == 1 || j == 1 || i == N || j == N
            RHS_Lap(global_index(j, i, N)) = base_elevation_m / h^2;
        end
    end
```

```
end

reg_param = 1e-3;
LHSMat = [Lap, -Sib; Jib, -reg_param * speye(length(xib))];
RHSVec = [RHS_Lap; f2];
toc;

%% 6. DIRECT LINEAR SOLVE
fprintf('Solving sparse matrix system...\n');
tic;
ResVec = LHSMat \ RHSVec;
u = reshape(ResVec(1:N^2), N, N);
toc;

%% 7. VISUALIZATION
figure(1); set(gcf, 'Position', window_size_for_export);

% Plot 1: 3D Surface Solution with UTM and Elevation
subplot(1, 2, 1);
surf(X_utm, Y_utm, u, 'FaceAlpha', 0.85, 'EdgeColor', 'none');
hold on;
colormap(slanCM('plasma')); % Applied plasma colormap

for c = 1:num_contours
    plot3(xib_utm{c}, yib_utm{c}, ones(size(xib_utm{c})) * scaled_elevs(c), 'r-
       ', 'LineWidth', lwlrg/2);
end

xlabel('UTM Easting [m]', 'Interpreter', 'latex');
ylabel('UTM Northing [m]', 'Interpreter', 'latex');
zlabel('Elevation [m]', 'Interpreter', 'latex');
title(sprintf('PDE Topography Solution ($N=%d$)', N), 'Interpreter', 'latex');
fontsize(font, "points");
set(gca, 'TickLabelInterpreter', 'latex');
ax1 = gca; ax1.LineWidth = lwsml*.5;
view(-37.5, 30); % Restored 3D isometric view

% Plot 2: Top-down view of contour fit comparison
subplot(1, 2, 2);
hold on;
```

```
for c = 1:num_contours
    x_raw_utm = utm_easting_center + (x_scaled{c} - pi) * (1000 / (2*pi));
    y_raw_utm = utm_northing_center + (y_scaled{c} - pi) * (1000 / (2*pi));

    p1 = plot(x_raw_utm, y_raw_utm, 'k--', 'LineWidth', lwsml*.75);
    p2 = plot(xib_utm{c}, yib_utm{c}, 'r-', 'LineWidth', lw*.75);
end

plot(utm_easting_center, utm_northing_center, 'bx', 'MarkerSize', 15, '
   LineWidth', 3);

grid on; axis equal;
xlabel('UTM Easting [m]', 'Interpreter', 'latex');
ylabel('UTM Northing [m]', 'Interpreter', 'latex');
title('Contours vs. Fourier Epicycles', 'Interpreter', 'latex');
legend([p1, p2], {'Raw USGS Contours', sprintf('Fourier (%d Harmonics)',
   num_harmonics)}, 'Interpreter', 'latex', 'Location', 'northeast');
fontsize(font, "points");
set(gca, 'TickLabelInterpreter', 'latex');
ax2 = gca; ax2.LineWidth = lwsml*.5;

%% 8. HELPER FUNCTIONS
function g = global_index(j, i, N)
    g = N*(j-1) + i;
end

function [xc, yc, a_coeffs, b_coeffs, c_coeffs, d_coeffs] = contour_to_fourier(
   x_contour, y_contour, num_harmonics)
    x_contour = x_contour(:); y_contour = y_contour(:);
    M = length(x_contour);
    t = linspace(0, 2*pi, M+1)'; t(end) = [];

    xc = mean(x_contour); yc = mean(y_contour);
    a_coeffs = zeros(1, num_harmonics); b_coeffs = zeros(1, num_harmonics);
    c_coeffs = zeros(1, num_harmonics); d_coeffs = zeros(1, num_harmonics);

    for k = 1:num_harmonics
        a_coeffs(k) = (2/M) * sum(x_contour .* cos(k*t));
        b_coeffs(k) = (2/M) * sum(x_contour .* sin(k*t));
        c_coeffs(k) = (2/M) * sum(y_contour .* cos(k*t));
```

```
        d_coeffs(k) = (2/M) * sum(y_contour .* sin(k*t));
    end
end

function [x_out, y_out] = generate_fourier_epicycle(theta, xc, yc, a, b, c, d)
    x_out = xc * ones(size(theta)); y_out = yc * ones(size(theta));
    max_k = max([length(a), length(b), length(c), length(d)]);
    a(end+1:max_k) = 0; b(end+1:max_k) = 0;
    c(end+1:max_k) = 0; d(end+1:max_k) = 0;

    for k = 1:max_k
        x_out = x_out + a(k)*cos(k*theta) + b(k)*sin(k*theta);
        y_out = y_out + c(k)*cos(k*theta) + d(k)*sin(k*theta);
    end
end

exportgraphics(gcf,"HW2_4.jpg")
```

6 References

The original assignment description(s) are provided below along with scratch work.

HW 1: Working with Immersed Boundaries

In this mini-project assignment, we'll learn to work with **Immersed Boundary Methods (IBM)**, though technically we'll be using the **Force Coupling Method (FCM)** – the only difference is the kernel. You'll modify the 2D Poisson code from HW0 to account for an internal Dirichelet boundary condition. To do this, we'll implement a **Matrix-Free** linear solver with GMRES to solve the **Saddle Point System** that shows up.

Problem 1: Poisson Equation

First let's make sure our code from HW0 still works. Use a finite difference method to solve the Poisson equation $\nabla^2 u = f(x, y)$ in the domain $x \in [0, 2\pi], y \in [0, 2\pi]$ with Dirichelet boundary conditions on all sides. Choose f and the boundary data so that the test solution $u(x, y) = \cos(2x)\sin(2y)$ solves the problem. Then verify that your code converges with second order in the grid size Δx.

Problem 2: Poisson Equation with an Immersed Boundary

Now we'll consider the same setup in Problem 1, but with the additional Dirichelet condition:

$$u(x^{ib}, y^{ib}) = \cos\left(2x^{ib}\right)\sin\left(2y^{ib}\right), \tag{1}$$

$$x^{ib} = \pi + \cos(\theta), \quad y^{ib} = \pi + \sin(\theta), \quad \theta \in [0, 2\pi]. \tag{2}$$

Note that this **does not change the test solution from problem 1**, so we can still use it to check our work. To solve this problem using the IBM/FCM we'll use the following procedure

1. Discretize (x^{ib}, y^{ib}) using N_{ib} points so that the spacing between these IB points is roughly the same as the Δx from the finite difference grid.

   (a) **Once your code works, experiment with this spacing. What happens when the IB points are very far apart compared to the finite difference mesh size? What happens when they are very close together? Look at both the accuracy (Problem 3 will give a better idea of this) and the speed of your code?**

2. Imagine we placed a charge q_k on each IB point (x_k^{ib}, y_k^{ib}). We want to *solve* for those charges q_k so that the internal boundary condition $u(x^{ib}, y^{ib}) = \cos\left(2x^{ib}\right)\sin\left(2y^{ib}\right)$ is maintained. The addition of charges modifies our PDE to

$$\nabla^2 u = f(x, y) + \sum_{k=1}^{N_i b} q_k \delta_a \left(x - x_k^{ib}, y - y_k^{ib}\right) = f(x, y) + \boldsymbol{\mathcal{S}}^{ib}(\boldsymbol{q}),$$

   where we've introduced the compact notation $\boldsymbol{\mathcal{S}}$ for the *spreading* operation involving the regularized delta functions δ_a. **For this assignment, take δ_a to be a 2D Gaussian with variance $a = 1.2\Delta x$ (this is also what's used in the example code on canvas).**



$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x,y)$$

make sure eq for 2D Variance

$$u(x,y) = \cos(2x)\sin(2y)$$

$$\frac{\partial u}{\partial x} = -2\sin(2x)\sin(2y)$$

$$\frac{\partial^2 u}{\partial x} = -4\cos(2x)\sin(2y)$$

$$\frac{\partial u}{\partial y} = 2\cos(2x)\cos(2y)$$

$$\frac{\partial^2 u}{\partial y^2} = -4\cos(2x)\sin(2y)$$

$$-8\cos(2x)\sin(2y) = f(x,y)$$

leftover from problem 1

$$u(x,y) = \sin(x)\cos(y)$$

AI errors: Preallocation of matrix error in J fxn

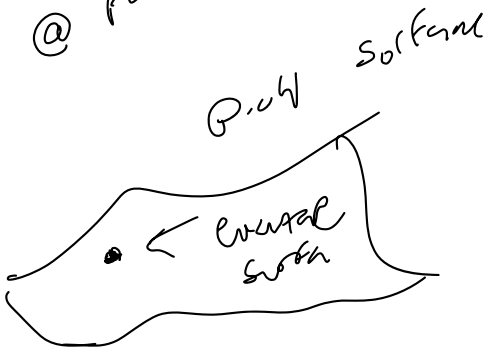
Sigma equation errors (res & term)

h^2 not scaled on S_{ij}

Write J first & test it with S will be similar

S_{ij} lets you plot point like curve in secant

J_{ij} given field find @ point



test interpolation operator on surface refinements

3. If we discretize our new PDE

$$\nabla^2 u = f(x,y) + \boldsymbol{\mathcal{S}}^{ib}(\boldsymbol{q})$$

directly, we'll have too many unknowns ($u(x_i, y_j$ and $\boldsymbol{q}$) and not enough equations. We can fix this by *interpolating* u onto the IB grid and enforcing our internal boundary condition

$$\iint_{(x,y)} u(x,y)\delta_a\left(x - x_k^{ib}, y - y_k^{ib}\right) dx\, dy = \boldsymbol{\mathcal{J}}^{ib}(u) = \cos\left(2x^{ib}\right)\sin\left(2y^{ib}\right).$$

4. After rearranging so that all of the unknowns are to one side, these equations together give the following **saddle point system**

$$\begin{bmatrix} \nabla^2 & -\boldsymbol{\mathcal{S}}^{ib} \\ \boldsymbol{\mathcal{J}}^{ib} & \mathbf{0} \end{bmatrix} \begin{bmatrix} u(x,y) \\ \boldsymbol{q} \end{bmatrix} = \begin{bmatrix} f(x,y) \\ \cos\left(2x^{ib}\right)\sin\left(2y^{ib}\right) \end{bmatrix}.$$

Discretize and solve this system

   (a) **Don't try to construct all of this matrix directly!** Write a routine to *apply* the matrix using our functions for $\boldsymbol{\mathcal{S}}^{ib}$ and $\boldsymbol{\mathcal{J}}^{ib}$ discussed in class and implemented in the example code. Then solve the system with GMRES

   (b) GMRES may have trouble converging with all of the default options. try using something like [what should 'tol' be?]

```
gmres(Amult, RHS, 1000, tol, 1)
```

   (c) **Don't forget:** when discretizing you'll have to modify the right hand side to account for the Dirichlet boundary data on the box $x \in [0, 2\pi], y \in [0, 2\pi]$.

5. Once you've solved this system, **verify that you solution is still converging in Δx, but the order should decrease to $\Delta x^{1.5}$ when the IB is included.**

6. If everything has gone according to plan, it should look like you haven't done anything at all – see Fig. 2

Problem 3: An almost minimal surface

Now that the code from Problem 2 works, let's use it to solve a problem we don't know the answer to. Solve the following PDE

$$\nabla^2 u = \boldsymbol{\mathcal{S}}^{ib}(\boldsymbol{q}), \tag{3}$$

$$u(0,y) = u(2\pi,y) = u(x,0) = u(x,2\pi) = 0 \tag{4}$$

$$u(x^{ib}, y^{ib}) = \cos\left(2x^{ib}\right)\sin\left(2y^{ib}\right), \tag{5}$$

$$x^{ib} = \pi + \cos(\theta), \quad y^{ib} = \pi + \sin(\theta), \quad \theta \in [0, 2\pi]. \tag{6}$$

This is just Problem 2 with $f(x,y) = 0$ and homogeneous Dirichlet boundary conditions. The solution should look something like Fig. 1. Note that the shape of $u(x,y)$ is closely related

(though not the same) as something called a minimal surface, which is the shape a soap film would make on a given wire-frame[1].

Problem 4: Charlie's rule

Charlie Peskin (the inventor of the IBM) says that you should never just stop one you have a cool code, **the best part is to play around with it.** So do something fun with your code and show it off! The following are a few ideas

1. The code for spreading $\boldsymbol{\mathcal{S}}^{ib}$ and interpolation $\boldsymbol{\mathcal{J}}^{ib}$ is extremely slow, but it doesn't need to be. The Gaussian kernel we use for δ_a is close to zero most of the time. Modify the code to *only* evaluate these functions when the value will be above some prescribed tolerance.

2. Try adding multiple shapes in the simulation to see what sort of surfaces you can get out of it

3. We can add *pseduo-dynamics* by moving our immersed boundary around or deforming it in time and re-solving for u. Make some fun videos of this.

4. GMRES performs much better with a good initial guess. Is there a way to make a good initial guess for problem 3? GMRES performs *even* better with an approximate inverse to the linear system, called a **preconditioner**. Read up on these and see if you have any ideas for this system (HINT: I didn't talk about Green's functions for no reason).

5. We have code to impose a Dirichelet condition on $u(x^{ib}, y^{ib})$. Can we modify the code to impose a Neumann condition on $\frac{\partial u(x^{ib},y^{ib})}{\partial n}$? (i.e can we prescribe the normal derivative on the immersed boundary)

[1] $\nabla^2 u = 0$ doesn't quite define a minimal surface. The actual PDE is

$$\text{Div}\left[\frac{\nabla u}{1+\|\nabla u\|^2}\right] = 0.$$

while non-linear equations are a bit harder, they're still doable and this may make for a nice project

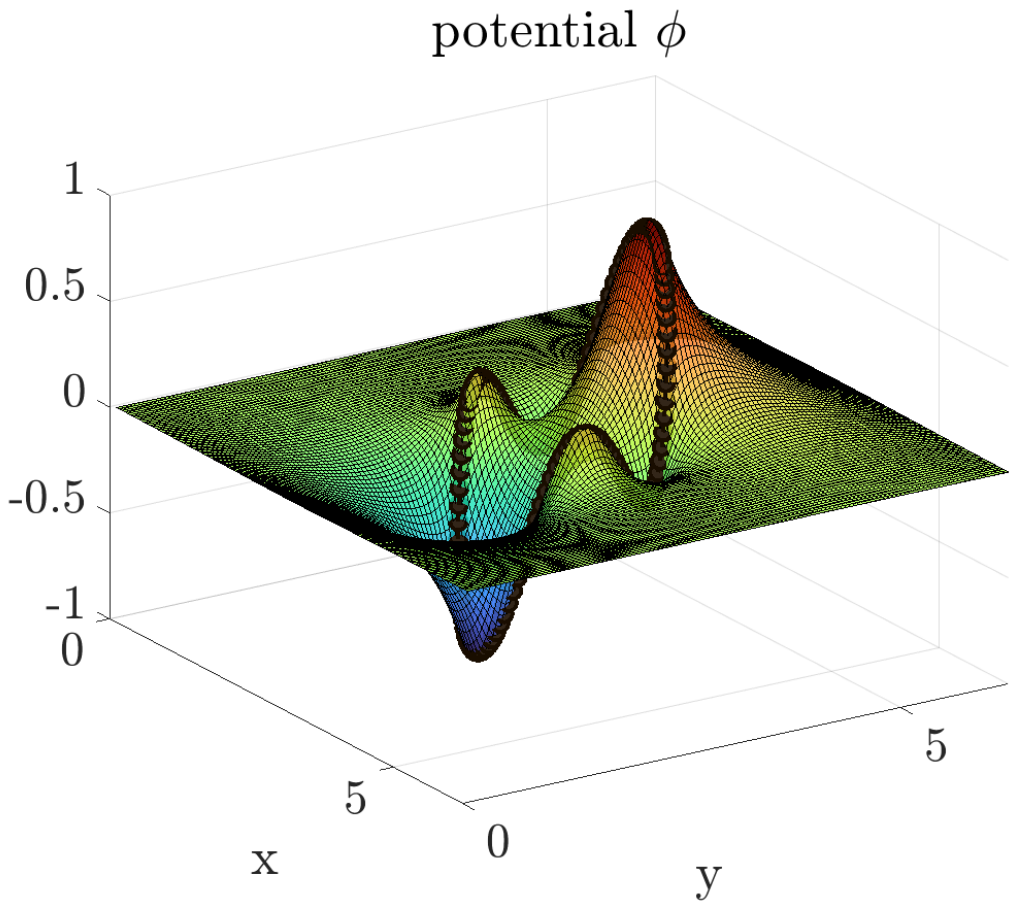


Figure 1: A plot of the solution $u(x, y)$ with the interpolated solution on the immersed boundary $u(x^{ib}, y^{ib})$, shown as a black line with markers representing the IB point locations.

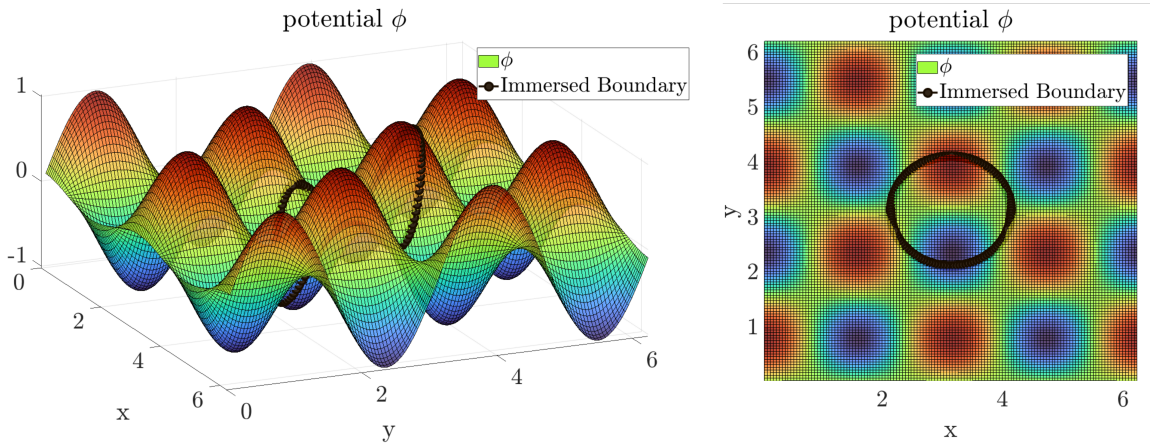


Figure 2: Left: A plot of the solution $u(x, y)$ with the interpolated solution on the immersed boundary $u(x^{ib}, y^{ib})$, shown as a black line with markers representing the IB point locations. Right: Top view of the left plot